

Generalized Anomaly Detection in Multi-Brand Industrial Control Systems Using Machine Learning

Based on the ICS-NAD real-world industrial control systems dataset, notebook workflow, model results, and cross-vendor generalization

Report Information

Item	Value
Dataset	ICS-NAD: real-world ICS network traffic from ABB, Siemens, and Schneider environments
Project Type	Industrial Control System Network Attack Detection
Vendors	ABB, Schneider, Siemens
Prediction Tasks	Multi-class, Binary, Category-level
Methods	ML, SMOTETomek, Feature Selection, Cross-Validation, Explainability
Prepared for	Capstone project report
Prepared by	Sahil
Mentor	Professor Rozhin Yasaei
Date	May 09, 2026
Course	INFO 698
Student Id	23961235

This report presents the machine learning project for detecting network attacks in Industrial Control System traffic. It includes the core methodology, dataset preparation, feature selection, class balancing, model comparison, important tables, diagrams, and final interpretation.

Table of Contents

1. Executive Summary
2. Dataset Overview
3. Project Objectives
4. Methodology and Pipeline Design
5. Data Cleaning and Preparation
6. Feature Engineering and Feature Selection
7. Class Balancing Using SMOTETomek
8. Models Evaluated
9. Experimental Setup
10. Final Results - Multi-Class Detection
11. Final Results - Binary Detection
12. Final Results - Category Detection
13. Best Model Summary
14. Essential Visual Results
15. Discussion of Findings
16. Explainability and Validation
17. Cross-Vendor Generalization
18. Limitations
19. Future Improvements
20. Conclusion
21. References

1. Executive Summary

Industrial Control Systems support critical infrastructure such as power generation, water treatment, manufacturing, and process automation. As these systems become more connected, they are exposed to cyber threats including reconnaissance, denial-of-service attacks, false data injection, and man-in-the-middle attacks.

This project applies machine learning to detect attacks in ICS network traffic using the ICS-NAD dataset. The completed pipeline processed traffic records from ABB, Schneider, and Siemens environments and evaluated several machine learning and neural models.

Three detection tasks were completed. The multi-class task predicts the exact attack type, the binary task detects benign versus attack traffic, and the category task groups traffic into broader attack categories. The final results show that tree-based methods, especially Random Forest and Decision Tree, performed the strongest overall.

A major challenge in this project is dataset scale. The full ICS-NAD collection is very large, and the processed workflow discussed in the notebook streamed more than 52 million records during the combine stage. A normal pandas-only workflow that loads all CSV files at once is not realistic on a typical local workstation. The final design therefore uses chunk-based loading, type downcasting, per-vendor Parquet files, and sampled training data. This makes the project reproducible and prevents repeated memory failures while still preserving the full combined dataset for future large-scale experiments

Table 1. Best Model Per Task

Task	Model	Accuracy	Precision	Recall	F1 Score
binary	Tuned_RF	0.9528	0.9567	0.9528	0.9533
category	DT	0.9993	0.9994	0.9993	0.9993
multi	RF	0.9203	0.9609	0.9203	0.9333

➤ Introduction and Problem Context

Industrial Control Systems, or ICS, are used to monitor and control physical processes in important sectors such as power generation, water treatment, manufacturing, transportation, and oil and gas. Unlike regular IT networks, ICS environments are directly connected to physical devices, including sensors, actuators, controllers, and supervisory systems. Because of this connection, a cyberattack on an ICS network can have consequences beyond data loss. It may cause operational downtime, equipment damage, unsafe conditions, financial losses, and even risks to human safety.

In the past, many ICS networks were kept separate from external systems. However, with the growth of Industry 4.0, these networks are now more connected to enterprise systems, cloud platforms, remote maintenance tools, and data analytics services. This increased connectivity improves monitoring, automation, and efficiency, but it also creates more opportunities for attackers. As a result, ICS networks are now exposed to threats such as denial-of-service attacks, reconnaissance,

spoofing, false data injection, man-in-the-middle attacks, and unauthorized command manipulation.

Protecting ICS environments is difficult because availability, reliability, and safety are usually the top priorities. Unlike normal IT systems, industrial devices often cannot be restarted, patched, or replaced frequently without disrupting operations. Many of these systems also use legacy protocols that were not originally designed with modern cybersecurity threats in mind. For this reason, intrusion detection plays an important role in ICS security. It allows network activity to be monitored and suspicious behavior to be detected without interrupting the physical process.

Machine learning can be useful for detecting attacks in ICS networks because network traffic often follows measurable patterns. These patterns may include packet counts, byte counts, flow duration, timing behavior, protocol usage, TCP flags, and source or destination activity. By learning from these features, machine learning models can help distinguish between normal behavior and malicious activity. However, building an effective detection model requires more than simply training a classifier. The dataset must be cleaned, labels must be checked for consistency, class imbalance must be addressed, relevant features must be selected, and performance must be evaluated using metrics such as accuracy, precision, recall, and F1-score.

This project uses the ICS-NAD dataset, which contains real-world ICS network traffic from multiple vendors, including ABB, Schneider, and Siemens. The project develops a complete machine learning workflow for three detection tasks: multi-class attack detection, binary benign-versus-attack detection, and category-level attack classification. By using realistic multi-vendor data, SMOTETomek balancing, Random Forest feature selection, and multi-model evaluation, the project shows how machine learning can support accurate and practical intrusion detection in industrial control environments.

2. Dataset Overview

The ICS-NAD dataset contains network traffic records collected from real-world Industrial Control System environments. The project used extracted CSV feature files instead of raw PCAP files because the CSV version is more suitable for supervised machine learning.

The dataset contains traffic from three industrial vendors: ABB, Schneider, and Siemens. These vendors represent different ICS environments and communication patterns, which makes the dataset useful for evaluating intrusion detection models.

Table 2. Dataset Processing Summary

Item	Value
CSV files processed	61
Vendors	ABB, Schneider, Siemens
Detected label column	state
Rows before total cap	1,199,754

Rows after total cap	800,000
Rows after unknown-label removal	785,439
Duplicates removed	2,916
Final cleaned rows	782,523
Final cleaned columns	62
Numeric feature columns	51
Categorical feature columns	8
Train-test split	60% training / 40% testing
Total runtime	678.45 minutes

The pipeline found 61 CSV files and detected the target label column as state. After applying a total row cap for memory safety, the data was cleaned and reduced to 782,523 final records.

3. Project Objectives and Scope

- **Build a complete machine learning workflow for ICS network attack detection.**
- **Use real-world ICS traffic from multiple vendors.**
- **Create three prediction tasks: multi-class, binary, and category-level detection.**
- **Handle class imbalance using SMOTETomek.**
- **Use Random Forest feature importance to select the most useful features.**
- **Train and compare multiple models using accuracy, precision, recall, and F1-score.**

The main objective of this project is to build a complete machine learning workflow for ICS network attack detection. The workflow includes data loading, cleaning, feature preparation, class balancing, feature selection, model training, evaluation, and final result reporting.

This project uses real-world ICS traffic from multiple vendors, including ABB, Schneider, and Siemens. Using multi-vendor data makes the project more realistic because industrial environments can differ in protocols, devices, and traffic behavior.

The project evaluates three prediction tasks: multi-class detection, binary detection, and category-level detection. Multi-class detection predicts the exact attack type, binary detection classifies traffic as benign or attack, and category-level detection groups attacks into broader categories.

Class imbalance is handled using SMOTETomek, which helps improve learning for minority attack classes by creating synthetic samples and cleaning overlapping records.

Random Forest feature importance is used to select the most useful features for each task. This improves efficiency, reduces unnecessary complexity, and makes the models easier to interpret.

Finally, multiple models are trained and compared using accuracy, precision, recall, and F1-score. These metrics help identify which model performs best for ICS network attack detection.

4. Methodology and Pipeline Design

The project followed a structured machine learning pipeline. First, the CSV files were loaded and combined. Next, the data was cleaned, labels were normalized, and three target variables were created. The data was then split into training and testing sets, scaled, balanced, and used to train multiple models.

The methodology was designed to be practical and reproducible. Each stage produced outputs that could be used in the final report, including result tables, plots, selected feature summaries, and saved model artifacts.

Table 3. Methodology Pipeline

Step	Stage	Description
1	Load ICS-NAD CSV files	Read vendor-specific CSV files and assign vendor labels
2	Clean and validate data	Remove invalid labels, duplicates, empty rows, and schema issues
3	Create targets	Generate multi-class, binary, and category labels
4	Split data	Use 60% training and 40% testing
5	Scale features	Normalize numeric features using MinMax scaling
6	Feature selection	Select top features using Random Forest importance
7	Balance data	Apply SMOTETomek on training data only
8	Train models	Evaluate several ML and DL models
9	Validate and analyze	Use CV, tuning, explainability, and error analysis

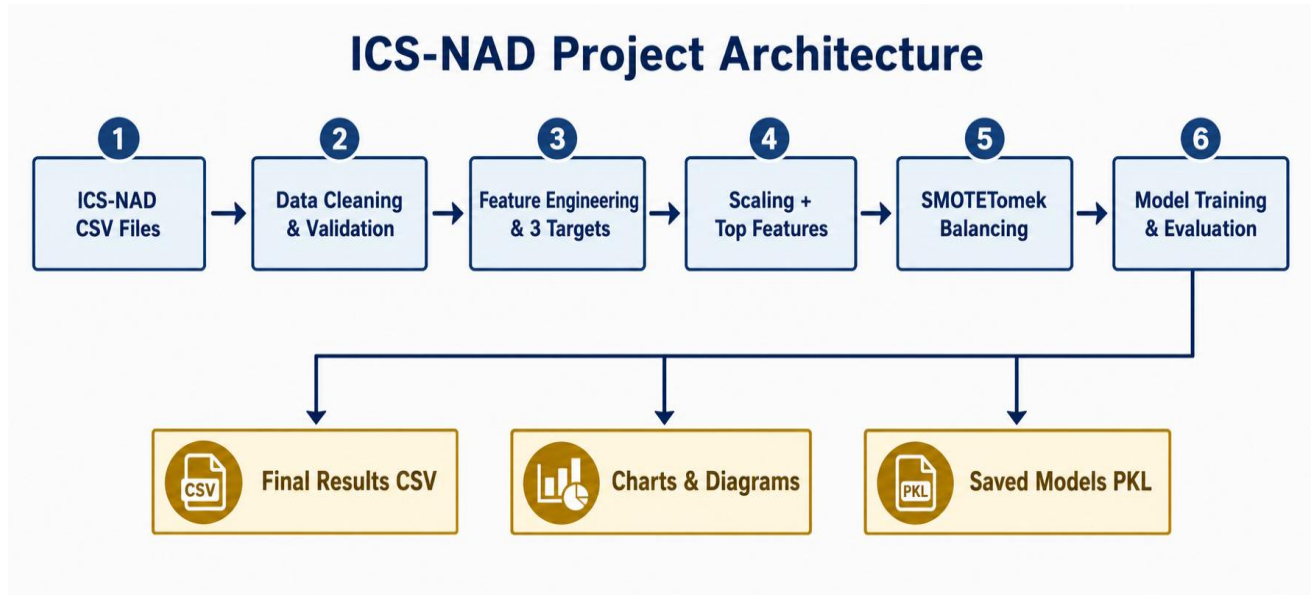


Figure 1. Overall Methodology Pipeline

5. Data Cleaning and Preparation

The cleaning stage ensured the data was suitable for machine learning. Rows with unknown labels were removed because supervised learning requires reliable target values. Exact duplicates were also removed to reduce bias and avoid repeated samples affecting training.

The final cleaned dataset had 782,523 rows and 62 columns. The pipeline identified 51 numeric columns and 8 categorical columns before modeling.

Table 4. Cleaning and Schema Validation Results

Cleaning Step	Result
Fully empty rows removed	0
Fully empty columns removed	0
Unknown-label filtering	Applied
Exact duplicates removed	2,916
Object columns converted to numeric	0
Final cleaned shape	782,523 rows x 62 columns

The preprocessing stage follows the logic recommended by the ICS-NAD paper while expanding it for local robustness. The paper describes removing constant columns, filling empty values, converting string values to numeric types, normalizing values to a 0-1 range, and splitting records into train/test subsets. The project extends this into a stronger, production-style preprocessing workflow

Preprocessing step	Reason for including it
Schema validation	Prevents training on malformed files or missing target fields.
Label normalization / inference	Creates a consistent Label column even when local CSV files do not include one directly.
Duplicate removal	Avoids inflating model performance with repeated records.
Missing and infinite value handling	Ensures models receive finite numeric inputs.
Outlier treatment	Reduces the effect of corrupted or extreme traffic values.
Low-variance removal	Drops features that add little useful information.
Collinearity reduction / VIF checks	Reduces redundant predictors and improves model stability.
Categorical encoding	Transforms protocol/state/vendor fields into numeric form.
MinMax scaling	Normalizes features so distance-based and neural models are not dominated by large-magnitude columns.
Feature selection	Keeps the most informative predictors and improves efficiency.
Artifact saving	Stores arrays, scalers, encoders, feature lists, and reports for reproducibility.

6. Feature Engineering and Feature Selection

The project created three target formats. The multi-class target keeps detailed attack labels, the binary target converts all attacks into one attack class, and the category target groups attacks into broader security categories.

Table 5. Prediction Task Summary

Task	Purpose	Comment
Multi	Predict exact attack label	Most difficult because it includes many attack classes
Binary	Classify benign vs attack	Useful for intrusion alerting
Category	Classify broader attack category	Groups attacks into Benign, DoS/DDoS, MitM, Other Attack, Reconnaissance

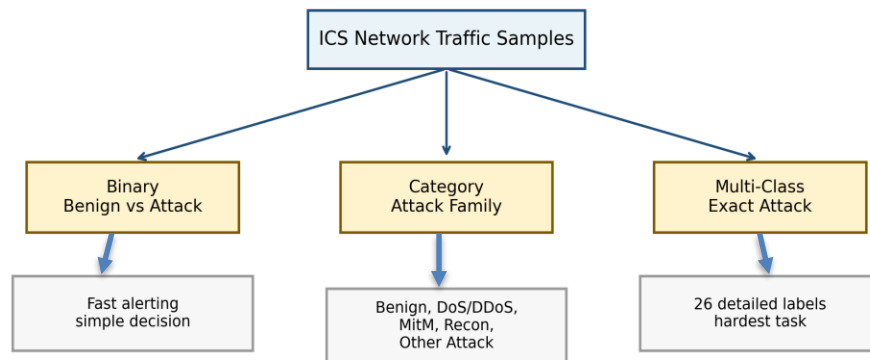
Feature selection was performed separately for each task. This is important because different prediction goals may depend on different traffic features.

Table 6. Selected Features by Task

Task	Selected Features	Interpretation
Multi	protocol, start, end, startOffset	Protocol behavior and flow timing
Binary	start, end, startOffset, endOffset	Timing and offset features separated benign vs attack traffic
Category	sAddress, protocol, start, end	Address, protocol, and timing separated broad attack categories

The selected features show that time-based features were consistently important. For the category task, source address and protocol were also selected, suggesting that network identity and protocol behavior help distinguish broader attack categories.

Three-Level Attack Detection Design

*Figure 2. Attack Detection Design*

7. Class Balancing Using SMOTETomek

Class imbalance was one of the main challenges in this project. Some classes had a very large number of samples while other attack classes had very few. Training directly on imbalanced data can cause models to favor majority classes and ignore rare attacks.

SMOTETomek was used to address this issue. SMOTE synthesizes minority-class samples, while Tomek links remove overlapping or ambiguous samples. This improves the balance of the training set while also cleaning decision boundaries.

Table 7. SMOTETomek Balancing Summary

Task	Before Balancing	After Balancing	Explanation
multi	469,513	approx. 4 million	Rare attack classes expanded heavily
binary	469,513	605,268	Attack and benign classes balanced to about 302k each
category	469,513	1,495,785	Five attack categories balanced close to 299k each

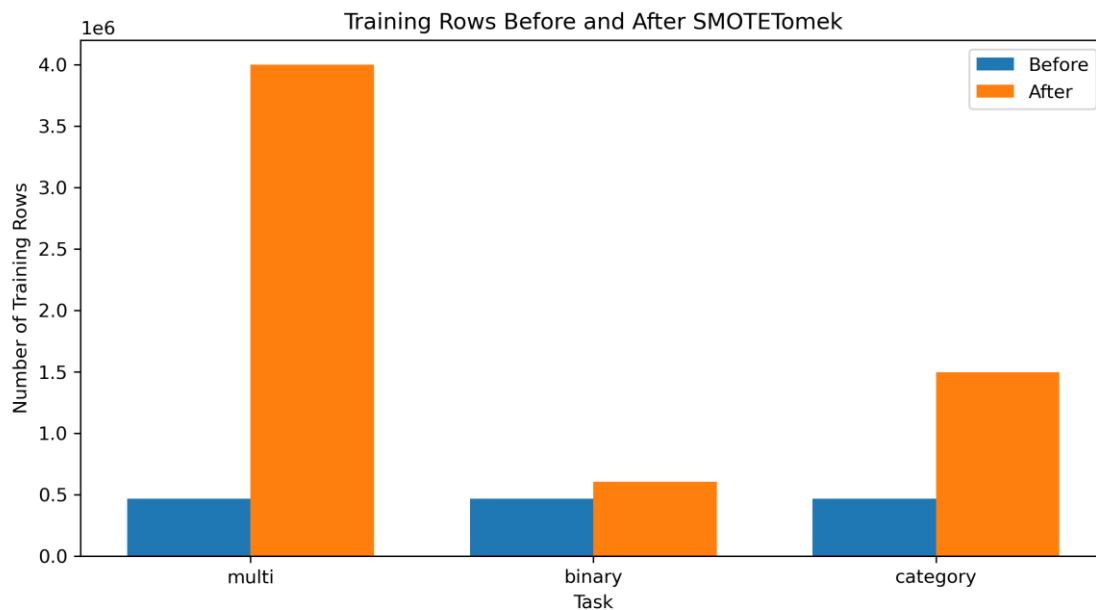


Figure 3. Training Rows Before and After SMOTETomek

The category task expanded to approximately 1.49 million rows after balancing. This explains why SVC and GBDT required longer runtime during model training.

8. Models Evaluated

Multiple model families were evaluated to compare different learning strategies. Tree-based models were included because they are strong for structured tabular data. Linear, probabilistic, distance-

based, boosting, support-vector, and neural-network models were also included for broader comparison.

Table 8. Models Used in the Project

Code	Model	Family	Observation
DT	Decision Tree	Tree-based	Fast and highly interpretable
RF	Random Forest	Tree ensemble	Strong general-purpose model for tabular data
KNN	K-Nearest Neighbors	Distance-based	Useful baseline but slower on large data
NB	Naive Bayes	Probabilistic	Fast baseline but weaker on this dataset
LR	Logistic Regression	Linear	Simple baseline; weaker on complex attack patterns
GBDT	Gradient Boosting Decision Tree	Boosting	Strong but can be slow
AB	AdaBoost	Boosting	Performed poorly in this project
SVC	Support Vector Classifier	Margin-based	Computationally expensive on large balanced data
ANN	Artificial Neural Network	Neural network	Useful comparison but did not outperform tree models
Tuned_RF	Tuned Random Forest	Optimized tree ensemble	Best binary task model

The final results show that tree-based models were generally more reliable than linear, probabilistic, SVM, and neural-network models for this feature representation.

9. Experimental Setup

The training and evaluation strategy used a 60/40 train-test split. Training data was balanced using SMOTETomek, while the test data was kept separate to represent unseen traffic.

The primary metrics were accuracy, precision, recall, and F1-score. F1-score is especially important because it balances precision and recall under class imbalance.

Table 9. Evaluation Metrics

Metric	Meaning
Accuracy	Overall proportion of correct predictions
Precision	How many predicted attacks/classes were correct
Recall	How many true attack/class samples were detected
F1-score	Balanced score between precision and recall

Table 10. Random Forest Hyperparameter Tuning Summary

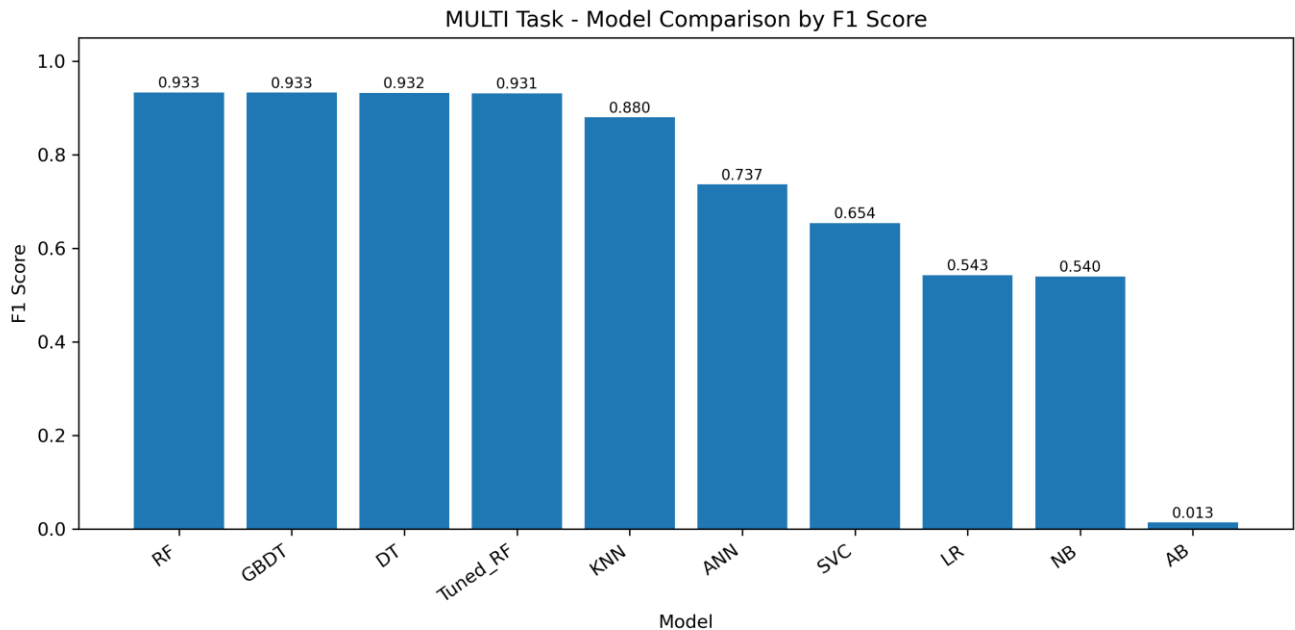
Task	Tuned Model	Best Parameters	F1 Score
multi	Tuned Random Forest	{'n_estimators': 200, 'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': None, 'max_depth': 30}	0.9315
binary	Tuned Random Forest	{'n_estimators': 150, 'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': 'sqrt', 'max_depth': 10}	0.9533
category	Tuned Random Forest	{'n_estimators': 150, 'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 'log2', 'max_depth': 40}	0.9991

10. Final Results - Multi-Class Attack Detection

The multi-class task predicts the exact attack class. This was the most difficult task because there were many attack labels and several rare classes. Random Forest produced the best F1-score among the main models, while GBDT and Decision Tree were also competitive.

Table 11. Multi-Class Model Results

Task	Model	Accuracy	Precision	Recall	F1 Score
multi	RF	0.9203	0.9609	0.9203	0.9333
multi	GBDT	0.9185	0.9645	0.9185	0.9326
multi	DT	0.9201	0.9567	0.9201	0.9320
multi	Tuned_RF	0.9172	0.9635	0.9172	0.9315
multi	KNN	0.8502	0.9458	0.8502	0.8798
multi	ANN	0.7014	0.8529	0.7014	0.7365
multi	SVC	0.6000	0.8192	0.6000	0.6539
multi	LR	0.5119	0.7311	0.5119	0.5426
multi	NB	0.6098	0.7365	0.6098	0.5396
multi	AB	0.0720	0.0078	0.0720	0.0135

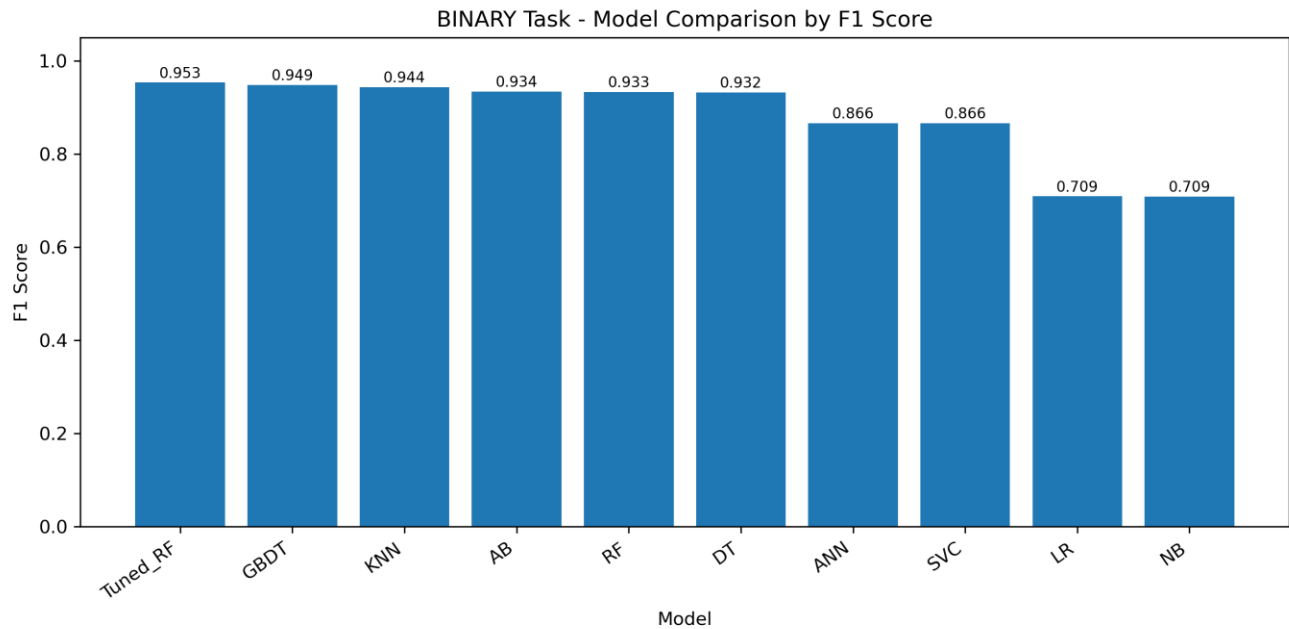
*Figure 4. Multi-Class Task F1-Score Comparison*

11. Final Results - Binary Attack Detection

The binary task classifies traffic as benign or attack. This is a practical intrusion detection setting because it answers whether a traffic sample is suspicious. The Tuned Random Forest achieved the best binary F1-score, followed closely by GBDT and KNN.

Table 12. Binary Model Results

Task	Model	Accuracy	Precision	Recall	F1 Score
binary	Tuned_RF	0.9528	0.9567	0.9528	0.9533
binary	GBDT	0.9478	0.9530	0.9478	0.9485
binary	KNN	0.9428	0.9474	0.9428	0.9435
binary	AB	0.9333	0.9401	0.9333	0.9344
binary	RF	0.9329	0.9336	0.9329	0.9332
binary	DT	0.9319	0.9325	0.9319	0.9321
binary	ANN	0.8663	0.8666	0.8663	0.8664
binary	SVC	0.8660	0.8661	0.8660	0.8661
binary	LR	0.7307	0.7190	0.7307	0.7089
binary	NB	0.7309	0.7192	0.7309	0.7088

*Figure 5. Binary Task F1-Score Comparison*

12. Final Results - Category-Level Attack Detection

The category task groups detailed attack labels into broader categories. This task achieved the strongest performance overall because broader categories are easier to separate than exact attack names.

Table 13. Category Model Results

Task	Model	Accuracy	Precision	Recall	F1 Score
category	DT	0.9993	0.9994	0.9993	0.9993
category	RF	0.9993	0.9994	0.9993	0.9993
category	Tuned_RF	0.9991	0.9992	0.9991	0.9991
category	KNN	0.9955	0.9960	0.9955	0.9956
category	GBDT	0.9918	0.9935	0.9918	0.9922
category	SVC	0.7836	0.9106	0.7836	0.8034
category	ANN	0.7472	0.8669	0.7472	0.7923
category	LR	0.6684	0.7741	0.6684	0.6217
category	NB	0.6345	0.6022	0.6345	0.5514
category	AB	0.2684	0.0996	0.2684	0.1445

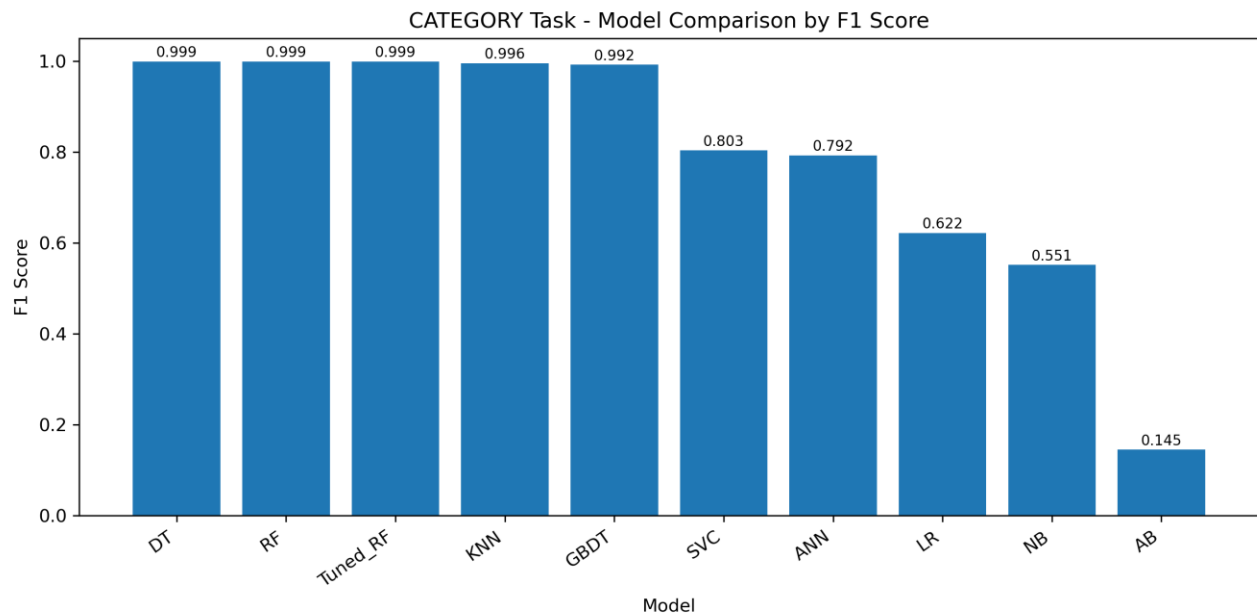


Figure 6. Category Task F1-Score Comparison

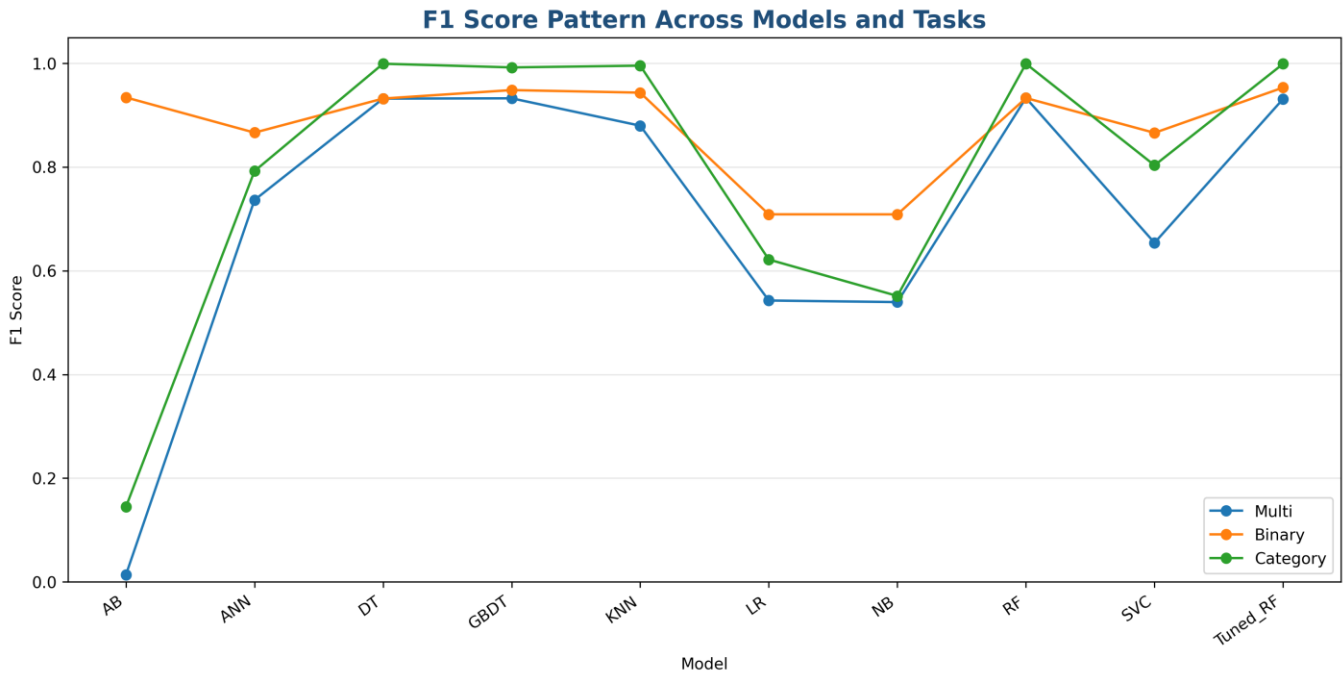


Figure 7. F1 score pattern graph across all the models used

13. Best Model Summary

The best model differed by task. This is expected because each task has a different prediction difficulty and class structure.

Table 14. Best Model Per Task

Task	Model	Accuracy	Precision	Recall	F1 Score
binary	Tuned_RF	0.9528	0.9567	0.9528	0.9533
category	DT	0.9993	0.9994	0.9993	0.9993
multi	RF	0.9203	0.9609	0.9203	0.9333

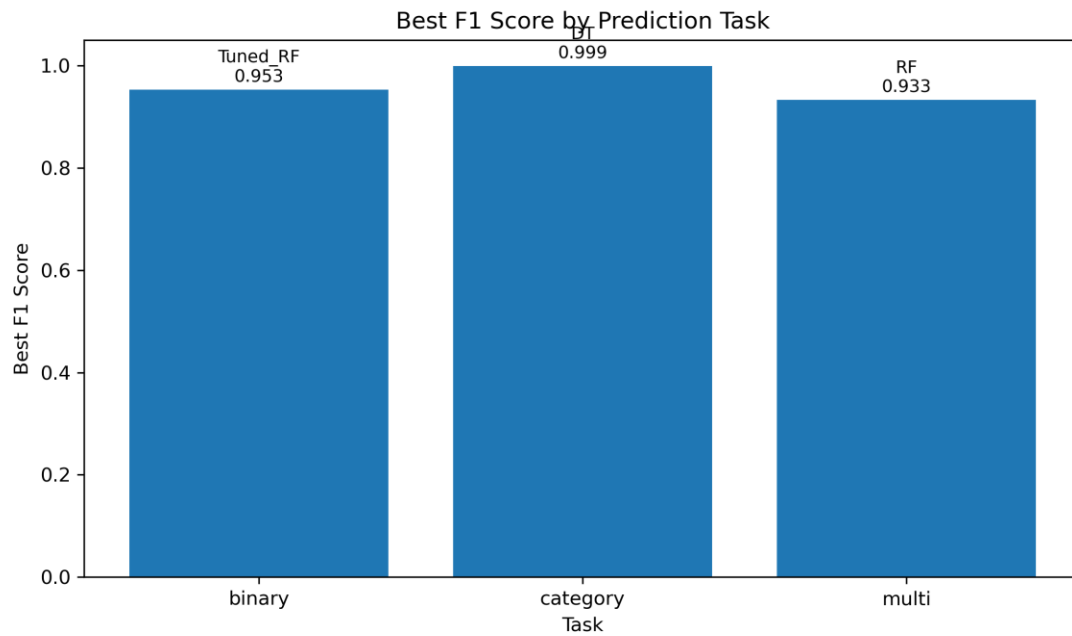


Figure 8. Best F1 Score by Task

For the binary task, the best model was Tuned_RF with accuracy 0.9528, precision 0.9567, recall 0.9528, and F1-score 0.9533.

For the category task, the best model was DT with accuracy 0.9993, precision 0.9994, recall 0.9993, and F1-score 0.9993.

For the multi task, the best model was RF with accuracy 0.9203, precision 0.9609, recall 0.9203, and F1-score 0.9333.

Best Model by Detection Task

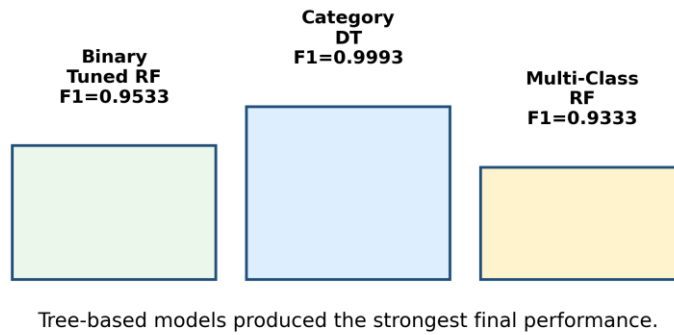


Figure 9. Best Model for Detection with F1 Scores

14. Visual Results

This section includes additional essential visualizations. These charts help compare model behavior across tasks and make the report more interpretable.

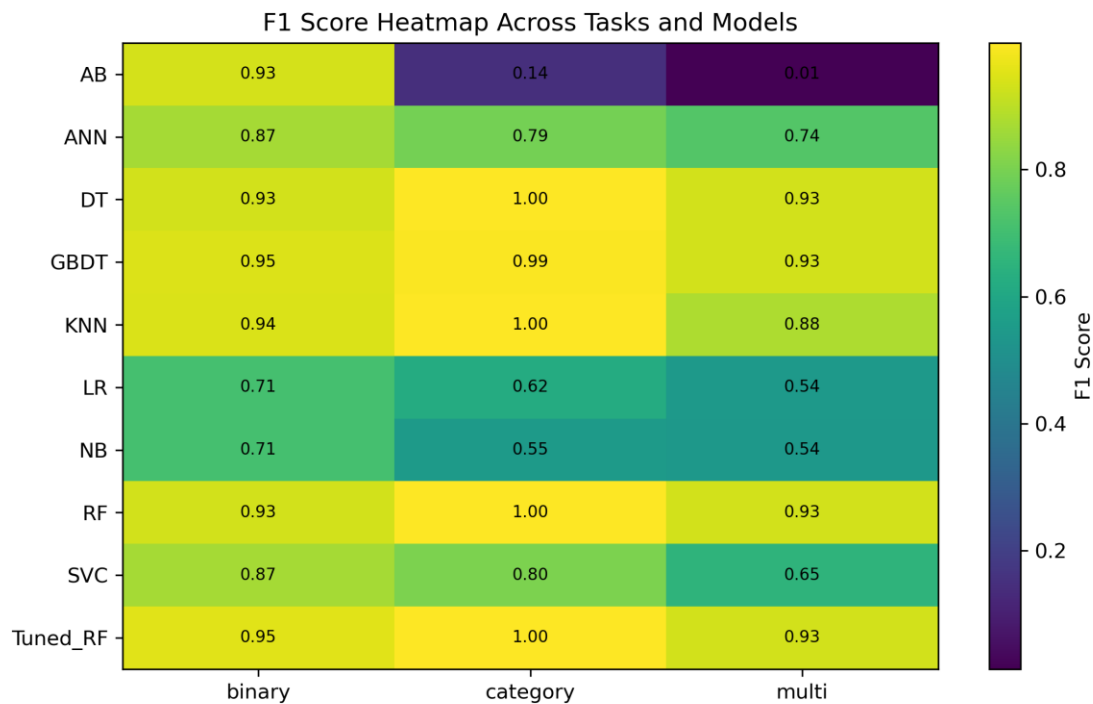


Figure 10. F1-Score Heatmap Across Tasks and Models

ICS-NAD Network Attack Detection Using Machine Learning

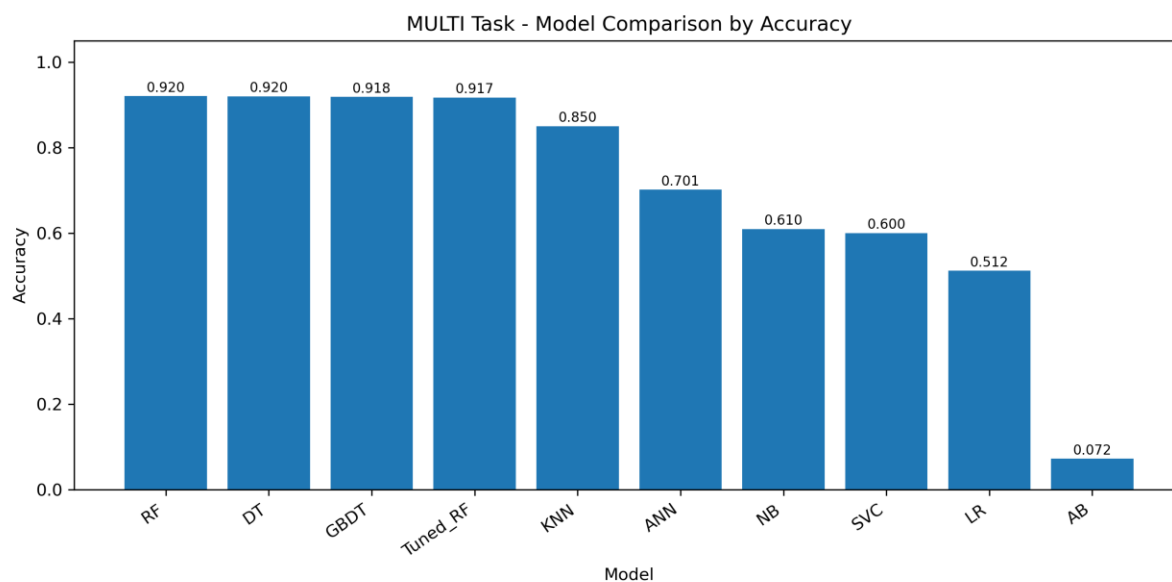


Figure 11. Multi-Class Accuracy Comparison

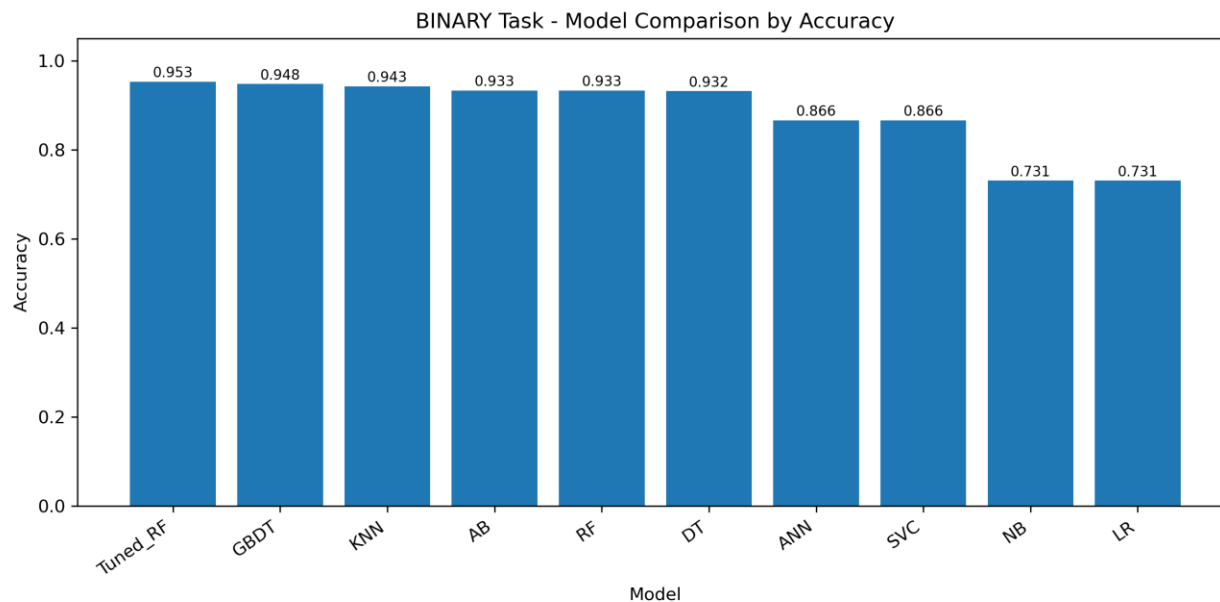


Figure 12. Binary Accuracy Comparison

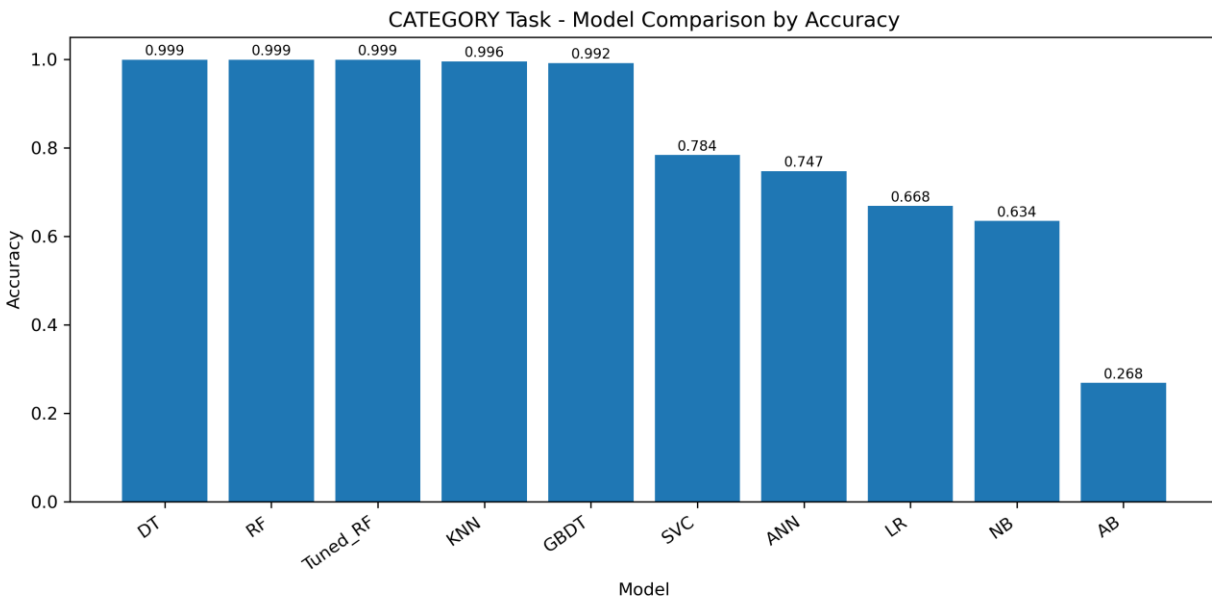


Figure 13. Category Accuracy Comparison

15. Discussion of Findings

The results show that the difficulty of the prediction problem changes depending on the target format. The multi-class task was the hardest because it required the model to distinguish between many detailed attack classes. The binary task was easier because all attacks were grouped into a single attack label. The category task was the easiest because it grouped attacks into broader categories.

Tree-based models performed best across the project. Random Forest achieved the strongest standard performance for the multi-class task, Tuned Random Forest performed best for binary classification, and Decision Tree performed best for category classification.

AdaBoost performed poorly on the multi-class and category tasks. This suggests that its weak learners and boosting behavior were not well matched with the selected feature representation. Naive Bayes and Logistic Regression were also weaker, likely because their assumptions are too simple for complex ICS attack patterns.

SVC and ANN provided useful comparisons but did not outperform tree-based models. SVC also required longer runtime, especially after SMOTETomek expanded the training data.

16. Explainability and Validation

The project included additional validation and explainability components. Stratified cross-validation was used to evaluate model stability. Feature importance and error-analysis outputs were generated to explain which features and classes affected performance.

- 5-fold stratified cross-validation helped preserve class proportions across folds.
- Random Forest feature importance was used for task-specific feature selection.
- Confusion matrix outputs can be used to inspect which attack classes were confused.
- Classification reports provide class-wise precision, recall, and F1-score.
- Permutation importance can strengthen explainability by measuring performance drops when features are shuffled.

17. Cross-Vendor Generalization

Cross-vendor testing evaluates whether a model trained on one vendor's traffic can generalize to another vendor's traffic. This is important because industrial environments differ by vendor, protocol, device behavior, and network topology.

The completed pipeline included cross-vendor experiment pairs for ABB, Schneider, and Siemens. This section is important for real-world ICS security because an intrusion detection model should ideally work beyond the specific vendor used during training.

Detailed cross-vendor CSV results were not found in the reports folder, but the pipeline log confirms that cross-vendor experiment sections were executed for all vendor pairs.

18. Limitations

- The pipeline used an 800,000-row cap for practical runtime and memory control.
- SMOTETomek improved class balance but greatly increased the training size.
- The category task became very large after balancing, which increased runtime for SVC and GBDT.
- The project used selected top features, which improved speed but may omit some useful information.
- Some rare attack classes were originally very small, making exact multi-class detection more difficult.
- ANN performance may improve with more architecture tuning, but it did not outperform tree-based methods in this run.

19. Future Improvements

- Run the pipeline on the complete dataset without row caps using a stronger workstation or cloud machine.
- Compare selected-feature models against models trained on all 60 extracted features.
- Add LightGBM, XGBoost, or CatBoost for faster high-performance boosting.
- Add SHAP explainability to interpret individual model predictions.
- Build a deployment script that loads the saved model and predicts new ICS traffic samples.
- Perform a deeper cross-vendor analysis to identify vendor-specific weaknesses.
- Create a real-time dashboard for alerting and monitoring model predictions.

20. Conclusion

This project successfully developed a complete machine learning workflow for detecting network attacks in Industrial Control System traffic using the ICS-NAD dataset. The pipeline covered all major stages, including data loading, cleaning, feature engineering, feature selection, class balancing, model training, evaluation, and result reporting. By using traffic from ABB, Schneider, and Siemens environments, the project was based on realistic multi-vendor ICS data rather than a simple simulated dataset.

The results show that tree-based models performed the strongest overall. Random Forest worked best for multi-class attack detection, Tuned Random Forest achieved the best result for binary attack detection, and Decision Tree performed best for category-level classification. This indicates that tree-based methods are highly suitable for structured ICS network traffic features because they can capture complex relationships between protocol, timing, address, and flow-based behavior.

SMOTETomek played an important role in improving the training process by reducing class imbalance and helping the models learn minority attack classes more effectively. Random Forest feature selection also helped reduce complexity by identifying the most useful features for each task. Overall, the project demonstrates that machine learning can be an effective approach for ICS intrusion detection when supported by careful preprocessing, balanced training data, proper feature selection, and strong model comparison.

The project demonstrates that careful preprocessing, class balancing, feature selection, and model comparison can produce strong intrusion detection performance for industrial network traffic.

References

1. Zhou, X., Cheng, Z., Wang, C., Wang, S., Tao, C., Zhou, Z., Chen, X., Luo, J., Wang, D., & Zhou, H. (2026). A dataset collected in real-world industrial control systems for network attack detection. *Scientific Data*, 13, 399. <https://doi.org/10.1038/s41597-026-06738-x>
2. Zhou, X. et al. (2025). A network attack detection dataset collected from multiple real-world industrial control systems. *Science Data Bank*. <https://doi.org/10.57760/sciencedb.25938>
3. Dehlaghi-Ghadim, A., Moghadam, M. H., Balador, A., & Hansson, H. (2023). Anomaly detection dataset for industrial control systems. *IEEE Access*, 11, 107982-107996.
4. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321-357.
5. Tomek, I. (1976). Two modifications of CNN. *IEEE Transactions on Systems, Man, and Cybernetics*, 6(11), 769-772.
6. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
7. National Institute of Standards and Technology. (2023). *Guide to Operational Technology (OT) Security: NIST Special Publication 800-82 Revision 3*. NIST.
8. Cybersecurity and Infrastructure Security Agency. (n.d.). *Cybersecurity Best Practices for Industrial Control Systems*. CISA.
9. Cybersecurity and Infrastructure Security Agency. (n.d.). *Industrial Control Systems*. CISA.
10. Alexander, O., Belisle, M., & Steele, J. (2020). *MITRE ATT&CK for Industrial Control Systems: Design and Philosophy*. MITRE.
11. Mathur, A. P., & Tippenhauer, N. O. (2016). *SWaT: A Water Treatment Testbed for Research and Training on ICS Security*.
12. Breiman, L. (2001). *Random Forests*. *Machine Learning*, 45, 5–32.
13. scikit-learn developers. (n.d.). *F1 score*. scikit-learn documentation.